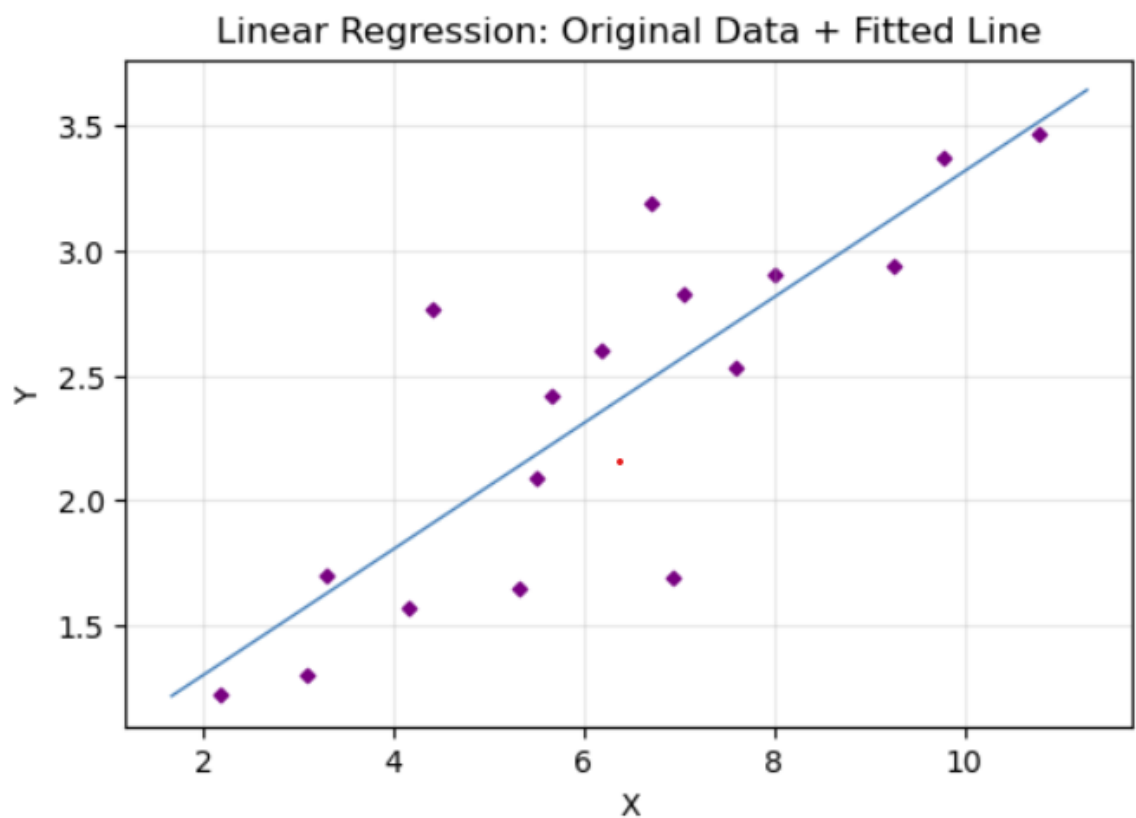


LAB Logbook

Lab 1



Lab 2

1. The accuracy of the improved model. Do not include any code or explanations in your lab logbook. Simply record the accuracy. For example, if the obtained accuracy is 0.98, then enter "0.98" in your lab logbook.

=> Model 1: 0.93

Model 2: 0.958

2. Copy your MLP architectures' screenshots to Logbook.

Model 1 MLP Architecture:

```
model = Sequential()
model.add(Dense(30, activation='relu', input_shape=(n_features,)))
model.add(Dense(24, activation='relu', input_shape=(n_features,)))

# Your code to add as many layers with activation functions of your choice:

# ...
model.add(Dense(18, activation='relu', input_shape=(n_features,)))
model.add(Dense(12, activation='relu', input_shape=(n_features,)))
model.add(Dense(6, activation='relu', input_shape=(n_features,)))
# ...

model.add(Dense(1, activation='sigmoid'))
```

```
print(model.summary())
```

Model: "sequential_8"

Layer (type)	Output Shape	Param #
dense_37 (Dense)	(None, 30)	1,050
dense_38 (Dense)	(None, 24)	744
dense_39 (Dense)	(None, 18)	450
dense_40 (Dense)	(None, 12)	228
dense_41 (Dense)	(None, 6)	78
dense_42 (Dense)	(None, 1)	7

Total params: 7,673 (29.98 KB)

Trainable params: 2,557 (9.99 KB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 5,116 (19.99 KB)

Model 2 Architecture:

```
model2 = Sequential([
    Dense(128, activation='relu', input_shape=(n_features,)),
    BatchNormalization(),
    Dropout(0.2),

    Dense(64, activation='relu', input_shape=(n_features,)),
    BatchNormalization(),
    Dropout(0.2),

    Dense(32, activation='relu'),
    BatchNormalization(),
    Dropout(0.2),

    Dense(16, activation='relu'),
    Dense(1, activation='sigmoid')
])

model2.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

```
print(model2.summary())
```

Model: "sequential_12"

Layer (type)	Output Shape	Param #
dense_56 (Dense)	(None, 128)	4,480
batch_normalization_4 (BatchNormalization)	(None, 128)	512
dropout_4 (Dropout)	(None, 128)	0
dense_57 (Dense)	(None, 64)	8,256
batch_normalization_5 (BatchNormalization)	(None, 64)	256
dropout_5 (Dropout)	(None, 64)	0
dense_58 (Dense)	(None, 32)	2,080
batch_normalization_6 (BatchNormalization)	(None, 32)	128
dropout_6 (Dropout)	(None, 32)	0
dense_59 (Dense)	(None, 16)	528
dense_60 (Dense)	(None, 1)	17

Total params: 47,877 (187.02 KB)

Trainable params: 15,809 (61.75 KB)

Non-trainable params: 448 (1.75 KB)

Optimizer params: 31,620 (123.52 KB)

Task 2 (Optional)

3. The output of the neural network as provided in Task 2.

```
# Your code of
# Keras Functional model
from tensorflow.keras.layers import Input, Dense, BatchNormalization, Dropout
from tensorflow.keras.models import Model
from tensorflow.keras.callbacks import EarlyStopping

# Input Layer
inputs = Input(shape=(n_features,))

# Block 1
x = Dense(128, activation='relu')(inputs)
x = BatchNormalization()(x)
x = Dropout(0.2)(x)

# Block 2
x = Dense(64, activation='relu')(x)
x = BatchNormalization()(x)
x = Dropout(0.2)(x)

# Block 3
x = Dense(32, activation='relu')(x)
x = BatchNormalization()(x)
x = Dropout(0.2)(x)

# Block 4
x = Dense(16, activation='relu')(x)

# Output layer
outputs = Dense(1, activation='sigmoid')(x)

# Build model
model_func = Model(inputs=inputs, outputs=outputs)

# Compile
model_func.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

Model: "functional_47"

Layer (type)	Output Shape	Param #
input_layer_13 (InputLayer)	(None, 34)	0
dense_61 (Dense)	(None, 128)	4,480
batch_normalization_7 (BatchNormalization)	(None, 128)	512
dropout_7 (Dropout)	(None, 128)	0
dense_62 (Dense)	(None, 64)	8,256
batch_normalization_8 (BatchNormalization)	(None, 64)	256
dropout_8 (Dropout)	(None, 64)	0
dense_63 (Dense)	(None, 32)	2,080
batch_normalization_9 (BatchNormalization)	(None, 32)	128
dropout_9 (Dropout)	(None, 32)	0
dense_64 (Dense)	(None, 16)	528
dense_65 (Dense)	(None, 1)	17

Total params: 16,257 (63.50 KB)

Trainable params: 15,809 (61.75 KB)

Non-trainable params: 448 (1.75 KB)

Lab 3

```
# Use model.add.Flatten() layer as the first layer.  
# Choose softmax in the output layer. Choose the number of layers and activation functions in  
# Do not forget to set a correct number of neurons in the output layer.
```

```
model = keras.Sequential([  
    keras.layers.Flatten(input_shape=(28, 28)),  
    keras.layers.Dense(128, activation='relu'),  
    keras.layers.Dense(64, activation='relu'),  
    keras.layers.Dense(8, activation='relu'),  
    keras.layers.Dense(10, activation='softmax')  
])
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
flatten_2 (Flatten)	(None, 784)	0
dense_8 (Dense)	(None, 128)	100,480
dense_9 (Dense)	(None, 64)	8,256
dense_10 (Dense)	(None, 8)	520
dense_11 (Dense)	(None, 10)	90

Total params: 328,040 (1.25 MB)

Trainable params: 109,346 (427.13 KB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 218,694 (854.28 KB)

Model Accuracy is 0.887

Lab 4

Your code to build MLP neural network model.

```
model = Sequential([
    Dense(128, activation="relu", input_shape=(X_train.shape[1],)),
    Dense(64, activation="relu"),
    Dense(32, activation="relu"),
    Dense(16, activation='relu'),
    Dense(8, activation='relu'),
    Dense(1) # regression output
])
```

Model: "sequential"

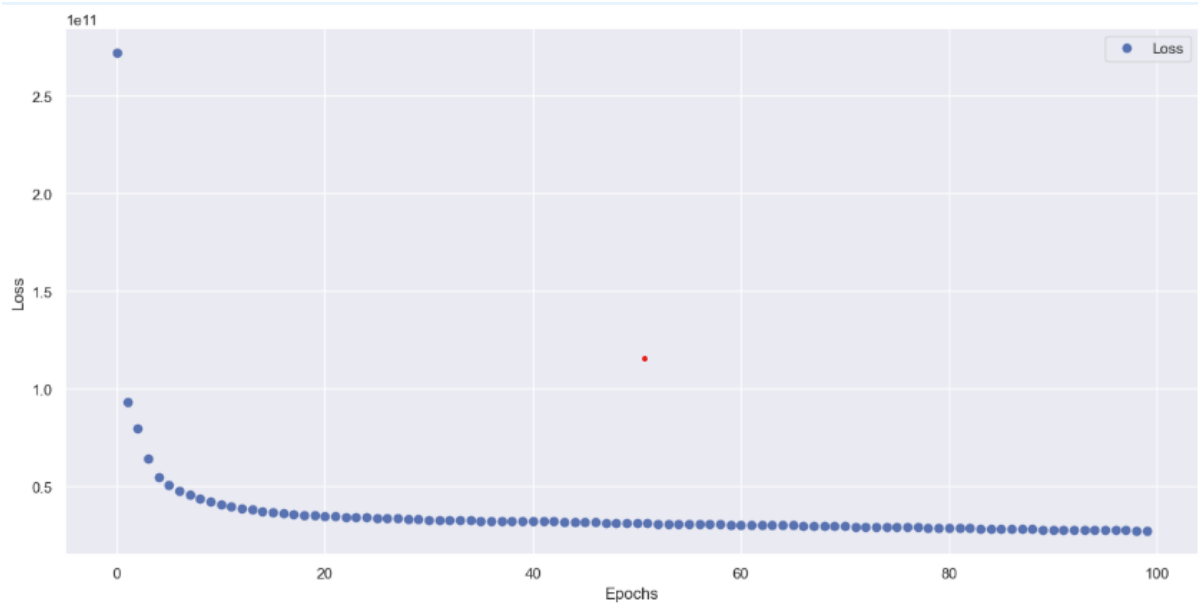
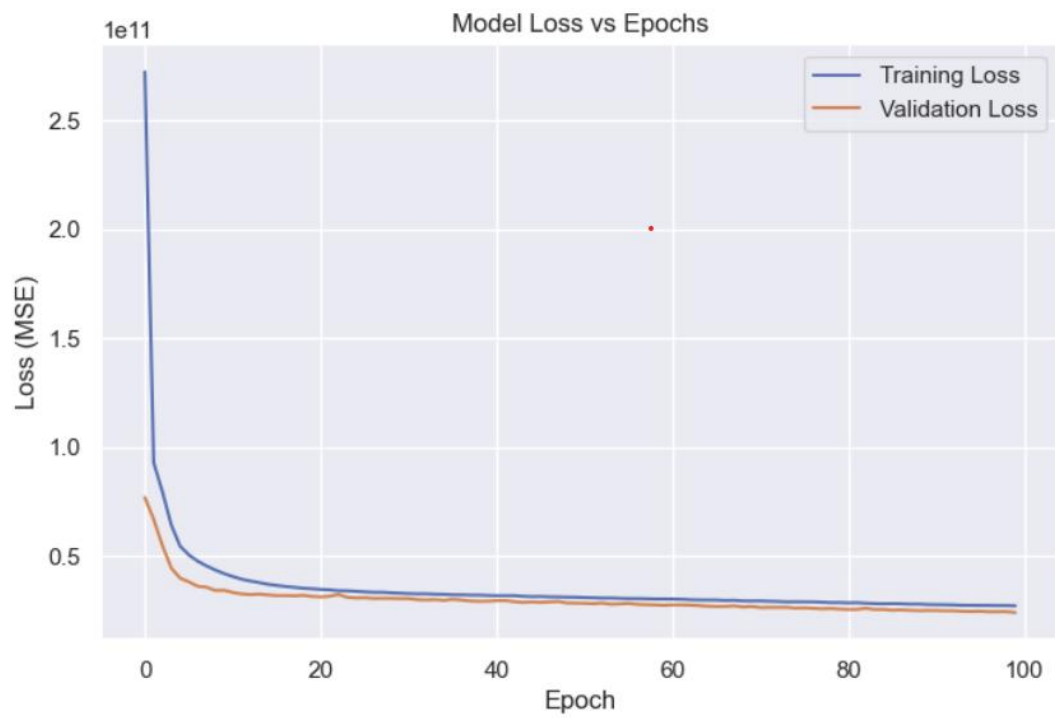
Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	2,560
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 32)	2,080
dense_3 (Dense)	(None, 16)	528
dense_4 (Dense)	(None, 8)	136
dense_5 (Dense)	(None, 1)	9

Total params: 40,709 (159.02 KB)

Trainable params: 13,569 (53.00 KB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 27,140 (106.02 KB)



Lab 5

1. Plot of the model building code and model's summary.

```
model = Sequential()

# CONVOLUTIONAL LAYER
model.add(Conv2D(filters=32, kernel_size=(4,4),input_shape=(32, 32, 3), activation='relu',))
# POOLING LAYER
model.add(MaxPool2D(pool_size=(2, 2)))

# Add more convolution and pooling layers with your choice of filters, strides, and activation function
# Conv + Pool 2
model.add(Conv2D(filters=64, kernel_size=(3,3), activation='relu'))
model.add(MaxPool2D(pool_size=(2, 2)))

# Conv + Pool 3
model.add(Conv2D(filters=128, kernel_size=(3,3), activation='relu'))
model.add(MaxPool2D(pool_size=(2, 2)))

model.add(Flatten())

# Add a few dense Layers.
# Dense Layers
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(64, activation='relu'))

# Output layer (10 classes)
model.add(Dense(10, activation='softmax'))
```

Model: "sequential_4"

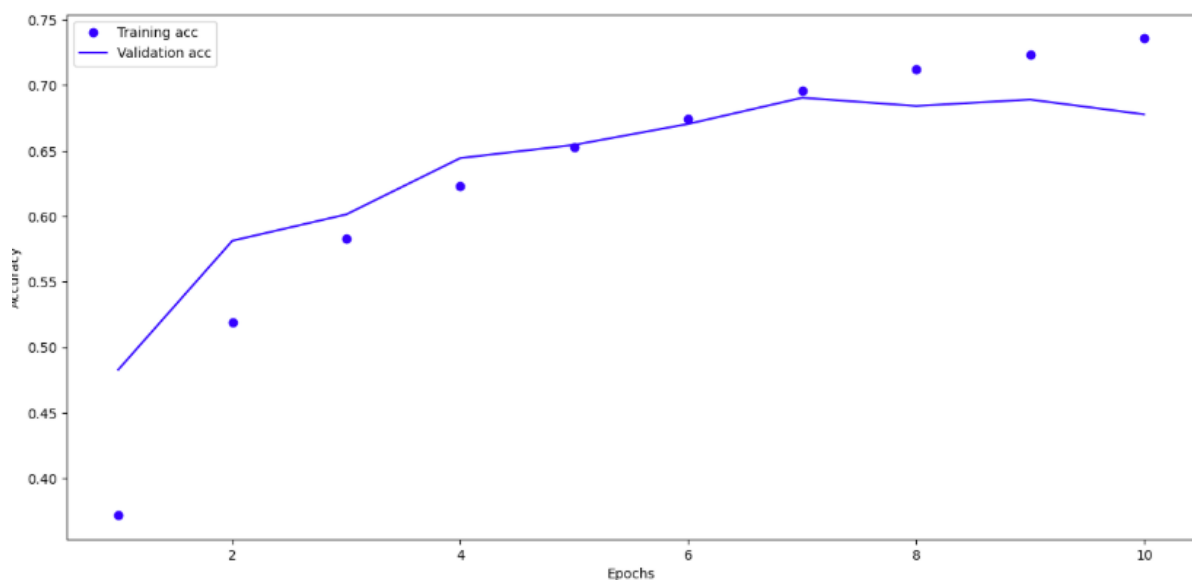
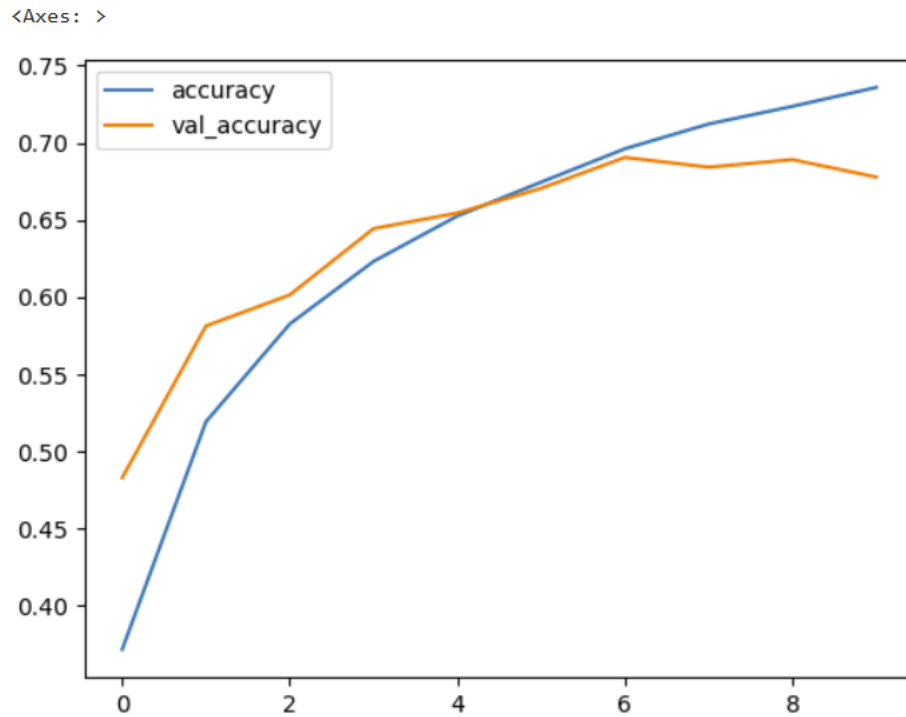
Layer (type)	Output Shape	Param #
conv2d_12 (Conv2D)	(None, 29, 29, 32)	1,568
max_pooling2d_12 (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_13 (Conv2D)	(None, 12, 12, 64)	18,496
max_pooling2d_13 (MaxPooling2D)	(None, 6, 6, 64)	0
conv2d_14 (Conv2D)	(None, 4, 4, 128)	73,856
max_pooling2d_14 (MaxPooling2D)	(None, 2, 2, 128)	0
flatten_4 (Flatten)	(None, 512)	0
dense_10 (Dense)	(None, 128)	65,664
dropout_3 (Dropout)	(None, 128)	0
dense_11 (Dense)	(None, 64)	8,256
dense_12 (Dense)	(None, 10)	650

Total params: 168,490 (658.16 KB)

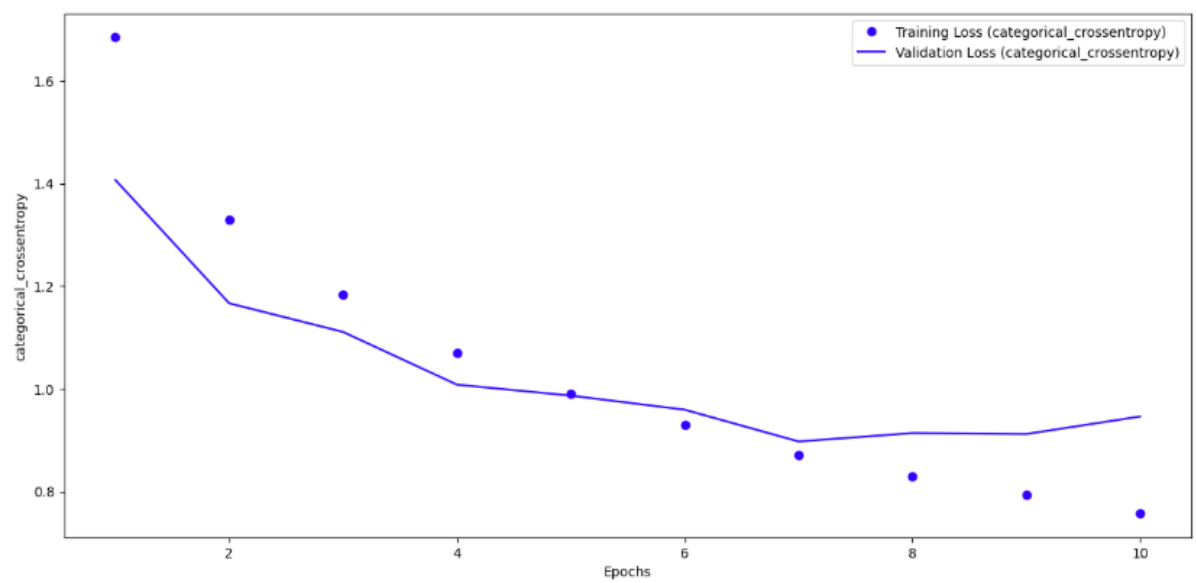
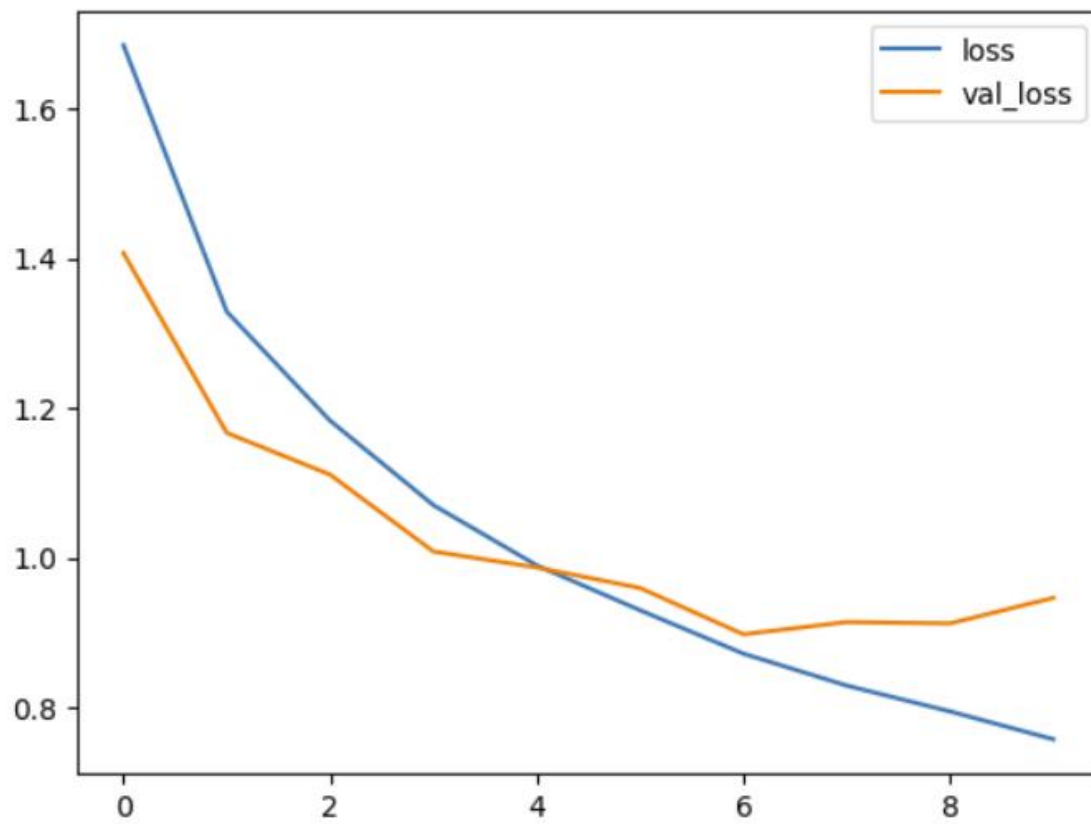
Trainable params: 168,490 (658.16 KB)

Non-trainable params: 0 (0.00 B)

2. Please ensure to document in your laboratory logbook a graphical representation depicting the train loss and validation loss trends (4 graphs).



<Axes: >



3. Based on this analysis, determine the optimal number of epochs for training your model.

=> The optimal number of epochs for training my model is 11.

Task 2 (Optional):

```
optimisers = ['SGD', 'RMSprop', 'Adagrad', 'Adadelta', 'Adam', 'Adamax', 'Nadam']
scores = []

for opt in optimisers:
    # Rebuild model each time (important!)
    model = Sequential()
    model.add(Conv2D(32, (4,4), input_shape=(32,32,3), activation='relu'))
    model.add(MaxPool2D((2,2)))
    model.add(Conv2D(64, (3,3), activation='relu'))
    model.add(MaxPool2D((2,2)))
    model.add(Flatten())
    model.add(Dense(128, activation='relu'))
    model.add(Dropout(0.5))
    model.add(Dense(10, activation='softmax'))

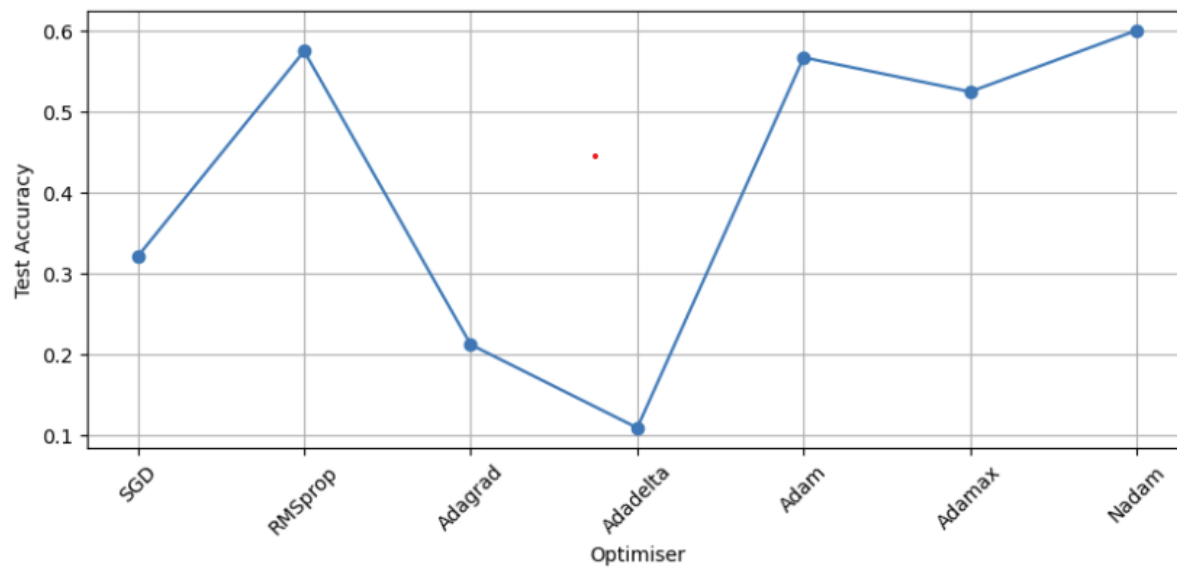
    model.compile(optimizer=opt, loss='categorical_crossentropy', metrics=['accuracy'])

    model.fit(x_train, y_cat_train, epochs=3, batch_size=128, validation_split=0.2, verbose=0)

    loss, acc = model.evaluate(x_test, y_cat_test, verbose=0)
    scores.append(acc)
    print(opt, ">", acc)

plt.figure(figsize=(10,4))
plt.plot(optimisers, scores, marker='o')
plt.xlabel("Optimiser")
plt.ylabel("Test Accuracy")
plt.xticks(rotation=45)
plt.grid(True)
plt.show()
```

SGD => 0.32100000977516174
RMSprop => 0.5752000212669373
Adagrad => 0.211899995803833
Adadelat => 0.10890000313520432
Adam => 0.5673999786376953
Adamax => 0.5249999761581421
Nadam => 0.6008999943733215



Lab 6

1. Record the 2nd model's code & plot the summary()

```
model = Sequential()  
model.add(GRU(64, activation='tanh', input_shape=(length, n_features)))  
model.add(Dropout(0.2))  
model.add(Dense(32, activation='relu'))  
model.add(Dense(1))
```

```
model.summary()
```

Model: "sequential_4"

Layer (type)	Output Shape	Param #
gru_2 (GRU)	(None, 64)	12,864
dropout (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 32)	2,080
dense_3 (Dense)	(None, 1)	33

Total params: 14,977 (58.50 KB)

Trainable params: 14,977 (58.50 KB)

Non-trainable params: 0 (0.00 B)

2. Copy your early-stop code and history_2 fit-training-process

```
SID = 2457431
last_digit = int(str(SID)[-1])

patience = last_digit + 10 if last_digit in [0, 1, 2, 3] else last_digit

early_stop = EarlyStopping(monitor='val_loss', patience=patience, restore_best_weights=True)
early_stop
```

fit the model

```
history_2 = model.fit(generator, epochs=30,
                       validation_data=validation_generator,
                       callbacks=[early_stop])
```

Epoch 1/30

384/384 ————— **4s** 4ms/step - loss: 0.0211 - val_loss: 0.0381

Epoch 2/30

384/384 ————— **1s** 4ms/step - loss: 0.0174 - val_loss: 0.0606

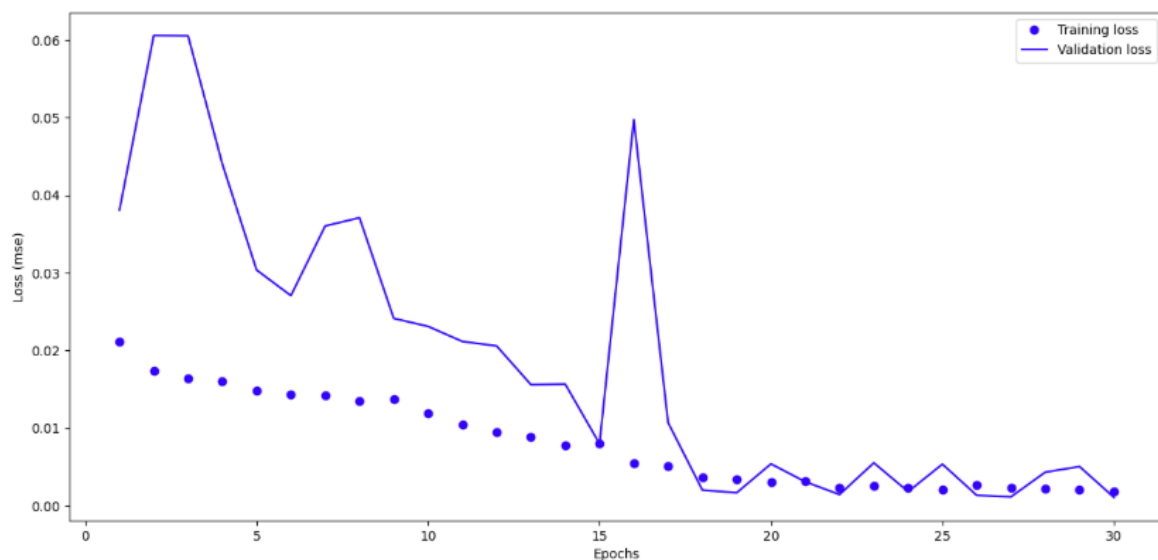
Epoch 3/30

384/384 ————— **1s** 4ms/step - loss: 0.0164 - val_loss: 0.0605

Epoch 4/30

384/384 ————— **3s** 4ms/step - loss: 0.0160 - val_loss: 0.0440

3. Record the plot of validation and test loss of history_2.

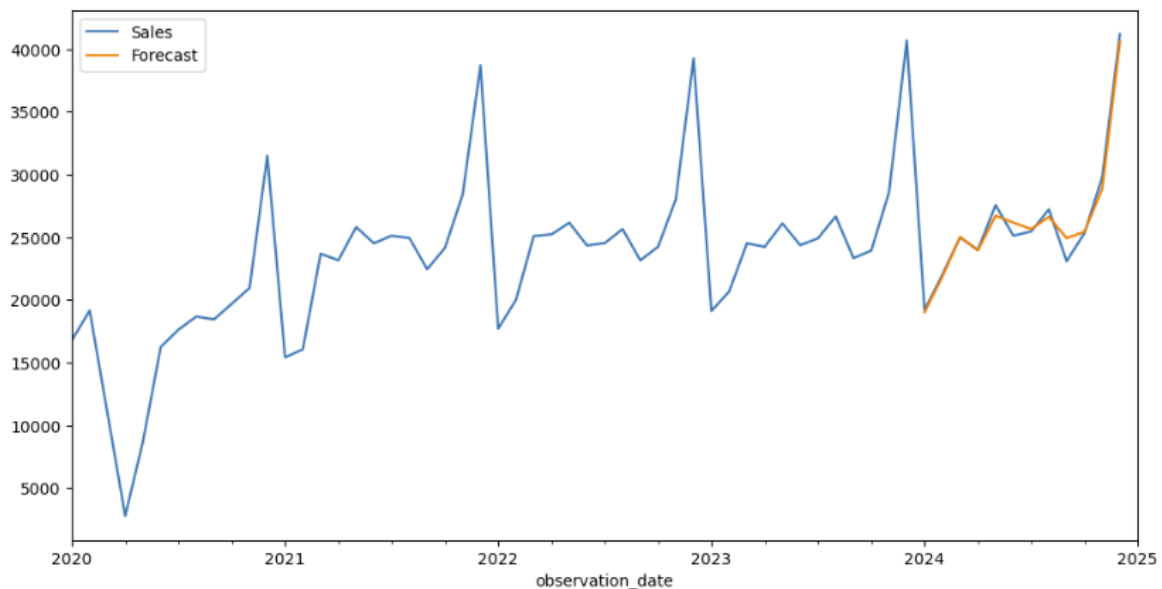


4. Based on this analysis, determine the optimal number of epochs for training your model.

```
import numpy as np
optimal_epoch = np.argmin(losses['val_loss']) + 1
print("Optimal number of epochs:", optimal_epoch)
```

Optimal number of epochs: 20

5. Record the plot of the Checking Forecast for the last 12 months (last graph).



Task 2 (Optional)

Try the same example with a LSTM and/or GRU! (Optional)

```
model_lstm = Sequential()
model_lstm.add(LSTM(64, activation='tanh', input_shape=(length, n_features)))
model_lstm.add(Dropout(0.2))
model_lstm.add(Dense(32, activation='relu'))
model_lstm.add(Dense(1))

model_lstm.compile(optimizer=Adam(learning_rate=0.001), loss='mse')
model_lstm.summary()
```

Model: "sequential_6"

Layer (type)	Output Shape	Param #
lstm_2 (LSTM)	(None, 64)	16,896
dropout_2 (Dropout)	(None, 64)	0
dense_6 (Dense)	(None, 32)	2,080
dense_7 (Dense)	(None, 1)	33

Total params: 19,009 (74.25 KB)

Trainable params: 19,009 (74.25 KB)

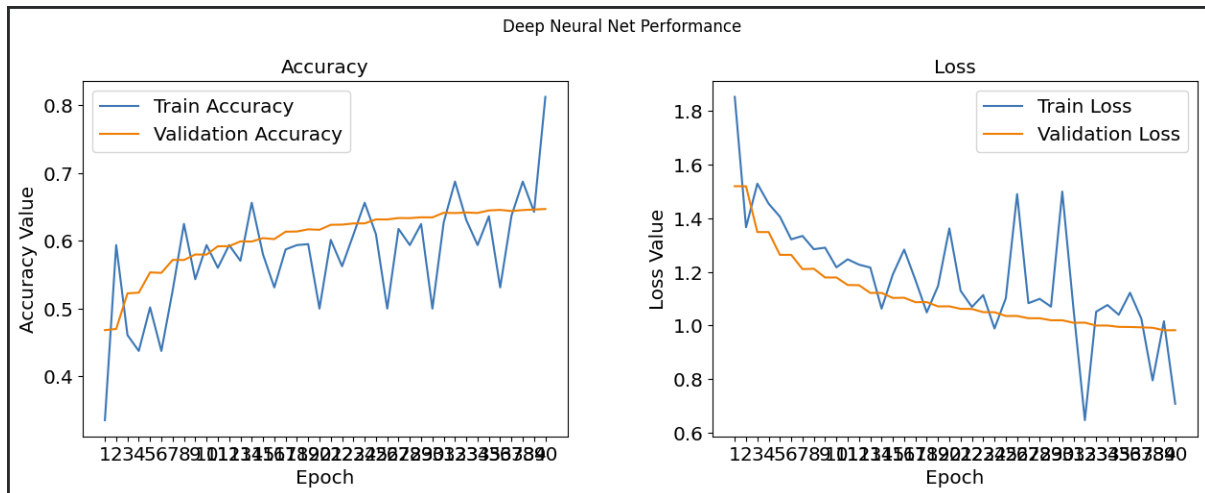
Non-trainable params: 0 (0.00 B)

Lab 7

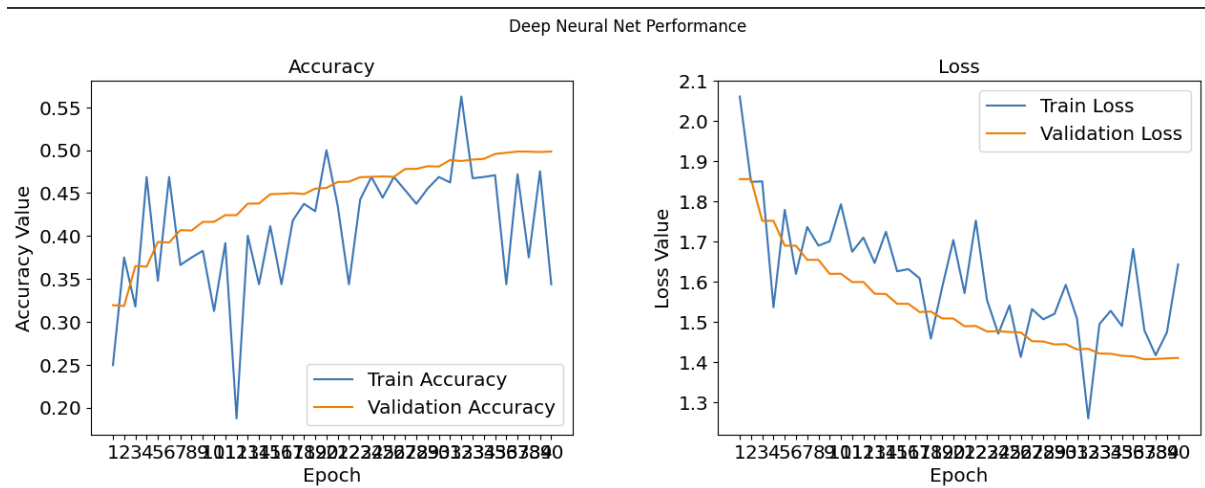
Task 1

1. Plot graphs for Accuracy and Loss Function for VGG16 and VGG19 (4 graphs in total).

VGG16



VGG19



2. Based on this analysis, determine the optimal number of epochs for training your VGG16 and VGG19 models.

=> Optimal epoch for (VGG16 = 40) and (VGG19 = 40)

3. Copy your early-stop code

```
from tensorflow.keras.callbacks import EarlyStopping

SID = 2457431 # replace with your student ID

last_digit = int(str(SID)[-1])

patience = 10 if last_digit == 0 else last_digit

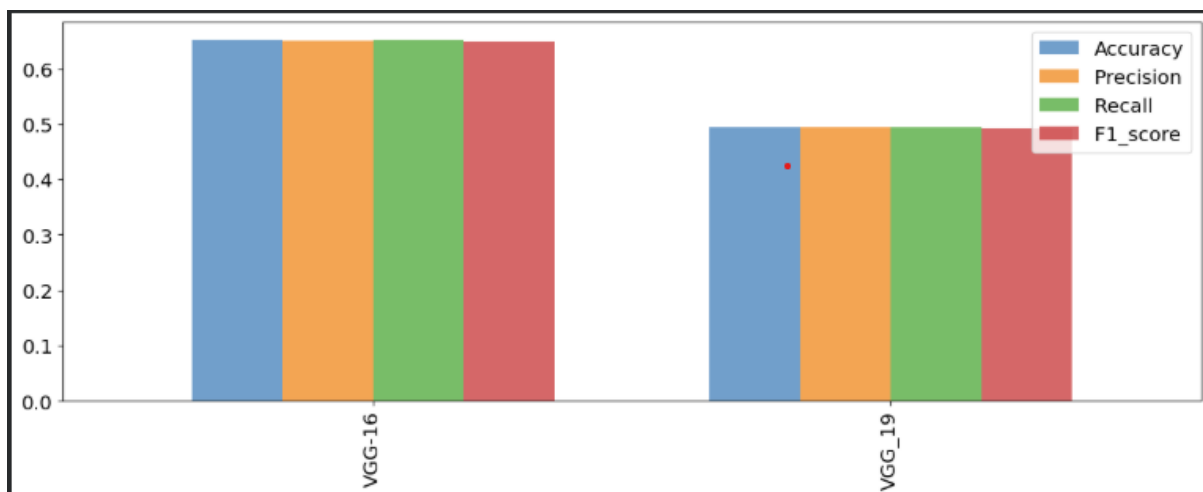
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=patience,
    restore_best_weights=True
)

early_stop
```

4. Add the final Performance Table

	Accuracy	Precision	Recall	F1_score
VGG-16	0.6558	0.651938	0.6558	0.652599
VGG_19	0.4957	0.489677	0.4957	0.490873

5. Plot the performance comparing histogram of VGG16 and VGG19



Task 2 (Optional)

1. Use the base_model 'Inception-V3' pre-trained neural network model.

```
base_model_3 = keras.applications.InceptionV3(
    weights='imagenet',
    include_top=False,
    input_shape=(75, 75, 3)
)

x = GlobalAveragePooling2D()(base_model_3.output)
x = BatchNormalization()(x)
x = Dense(256, activation='relu')(x)
x = Dense(256, activation='relu')(x)
x = Dropout(0.6)(x)
pred = Dense(NUM_CLASSES, activation='softmax')(x)

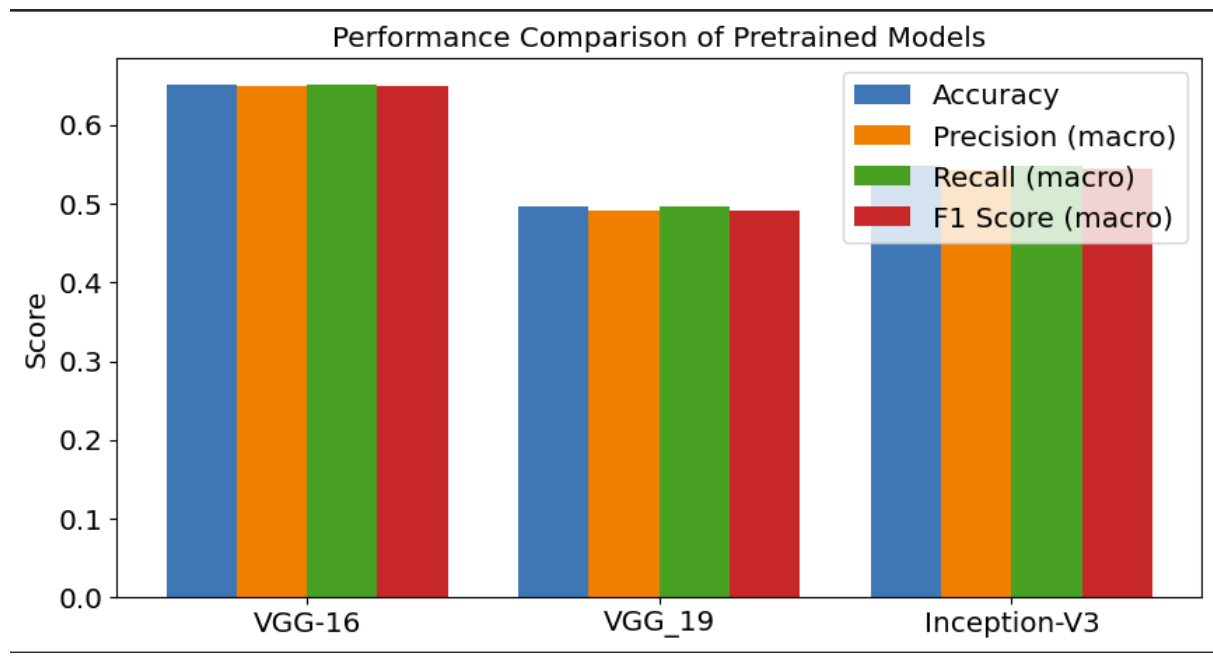
model_3 = Model(base_model_3.input, pred)

for layer in base_model_3.layers:
    layer.trainable = False

model_3.compile(
    loss='categorical_crossentropy',
    optimizer=optimizers.Adam(learning_rate=LEARNING_RATE),
    metrics=['accuracy']
)
```

2. Add Inception-V3 results to the Performance table and Performance comparing histogram.

	Model	Accuracy	Precision (macro)	Recall (macro)	F1 Score (macro)
0	VGG-16	0.6520	0.650358	0.6520	0.649906
1	VGG_19	0.4964	0.490435	0.4964	0.491456
2	Inception-V3	0.5473	0.542416	0.5473	0.543750



Lab 8

1) Your LSTM architecture using model.summary()

```
print(model_log.summary())
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
lstm_2 (LSTM)	(None, 41)	9,840
dense_2 (Dense)	(None, 2)	84

Total params: 9,924 (38.77 KB)

Trainable params: 9,924 (38.77 KB)

Non-trainable params: 0 (0.00 B)

None

2) Code and training result using model.fit()

```
history_log = model_log.fit(  
    X_train, y_train,  
    batch_size=20,  
    epochs=10,  
    validation_split=0.1,  
    verbose=1,  
    callbacks=[es_log, mc_log]  
)
```

Epoch 1/10

1425/1425 — 0s 6ms/step - loss: 0.0638 - mae: 0.0643

Epoch 1: val_loss improved from inf to 0.00003, saving model to best_model_logbook.keras

1425/1425 — 12s 7ms/step - loss: 0.0638 - mae: 0.0642 - val_loss: 3.4280e-05 - val_mae: 0.0041

Epoch 2/10

1422/1425 — 0s 5ms/step - loss: 5.6794e-05 - mae: 0.0056

Epoch 2: val_loss did not improve from 0.00003

1425/1425 — 8s 5ms/step - loss: 5.6799e-05 - mae: 0.0056 - val_loss: 2.2843e-04 - val_mae: 0.0142

Epoch 3/10

1422/1425 — 0s 5ms/step - loss: 6.4678e-05 - mae: 0.0061

Epoch 3: val_loss improved from 0.00003 to 0.00003, saving model to best_model_logbook.keras

1425/1425 — 8s 6ms/step - loss: 6.4673e-05 - mae: 0.0061 - val_loss: 2.5215e-05 - val_mae: 0.0035

Epoch 4/10

1417/1425 — 0s 5ms/step - loss: 7.5216e-05 - mae: 0.0066

Epoch 4: val_loss did not improve from 0.00003

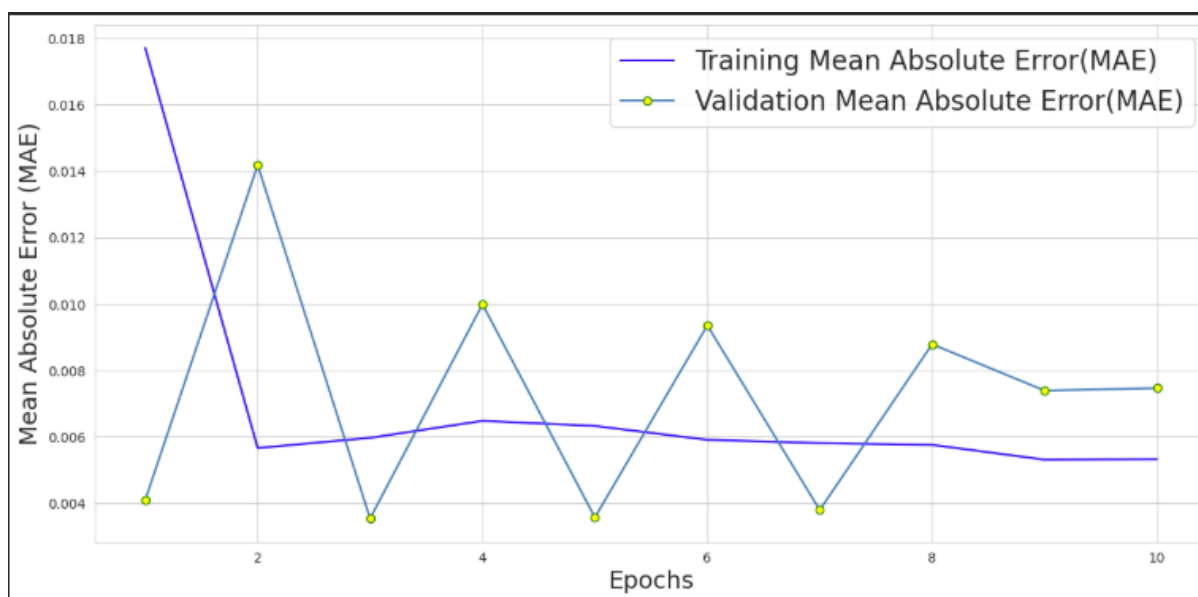
1425/1425 — 8s 6ms/step - loss: 7.5196e-05 - mae: 0.0066 - val_loss: 1.1529e-04 - val_mae: 0.0100

3) The resulting test MSE & MAE

```
scores_log = best_log_model.evaluate(X_test, y_test, verbose=1)  
scores_log
```

110/110 — 2s 7ms/step - loss: 2.8556e-05 - mae: 0.0037
[2.3433362002833746e-05, 0.0033884651493281126]

4) MAE detailed graph.



Lab 9

1. Code your Use EarlyStopping() and ModelCheckpoint() functions

```
es = EarlyStopping(monitor='val_loss', mode='min', patience=10, verbose=1)
mc = ModelCheckpoint('best_Trand-Flat_model_LSTM_Gold.keras', monitor='val_loss', mode='min', verbose=1, save_best_only=True)
```

2. code and training result using model.fit()

```
history_log = best_model.fit(
    X_train, y_train,
    epochs=epochs_log,
    batch_size=batch_size_log,
    validation_split=0.2,
    shuffle=True,
    verbose=1,
    callbacks=[es, mc],
    class_weight=class_weight_dict
)
```

Epoch 1/31
778/783 — 0s 8ms/step - accuracy: 0.5925 - loss: 0.6779
Epoch 1: val_loss improved from None to 0.53250, saving model to best_Trand-Flat_model_LSTM_Gold.keras
Epoch 1: finished saving model to best_Trand-Flat_model_LSTM_Gold.keras
783/783 — 9s 9ms/step - accuracy: 0.5639 - loss: 0.6814 - val_accuracy: 0.8168 - val_loss: 0.5325
Epoch 2/31
779/783 — 0s 8ms/step - accuracy: 0.5090 - loss: 0.6979
Epoch 2: val_loss did not improve from 0.53250
783/783 — 7s 9ms/step - accuracy: 0.5318 - loss: 0.6875 - val_accuracy: 0.3450 - val_loss: 0.7605
Epoch 3/31
782/783 — 0s 8ms/step - accuracy: 0.5602 - loss: 0.6651
Epoch 3: val_loss did not improve from 0.53250
783/783 — 7s 9ms/step - accuracy: 0.5379 - loss: 0.6702 - val_accuracy: 0.1446 - val_loss: 0.8542
Epoch 4/31
779/783 — 0s 8ms/step - accuracy: 0.5485 - loss: 0.6689
Epoch 4: val_loss did not improve from 0.53250
783/783 — 7s 9ms/step - accuracy: 0.5622 - loss: 0.6685 - val_accuracy: 0.3589 - val_loss: 0.7901
Epoch 5/31
779/783 — 0s 8ms/step - accuracy: 0.5328 - loss: 0.6562
Epoch 5: val_loss did not improve from 0.53250
783/783 — 7s 9ms/step - accuracy: 0.5601 - loss: 0.6610 - val_accuracy: 0.4403 - val_loss: 0.7657
Epoch 6/31
783/783 — 0s 8ms/step - accuracy: 0.5488 - loss: 0.6625
Epoch 6: val_loss did not improve from 0.53250
783/783 — 7s 9ms/step - accuracy: 0.5292 - loss: 0.6655 - val_accuracy: 0.7355 - val_loss: 0.6349
Epoch 7/31

3. the resulting test Accuracy and other error metrics for the classification task using `classification_report()`.

```
pred_probs_log = best_log_model.predict(X_test)
pred_log = (pred_probs_log >= 0.5).astype(int)

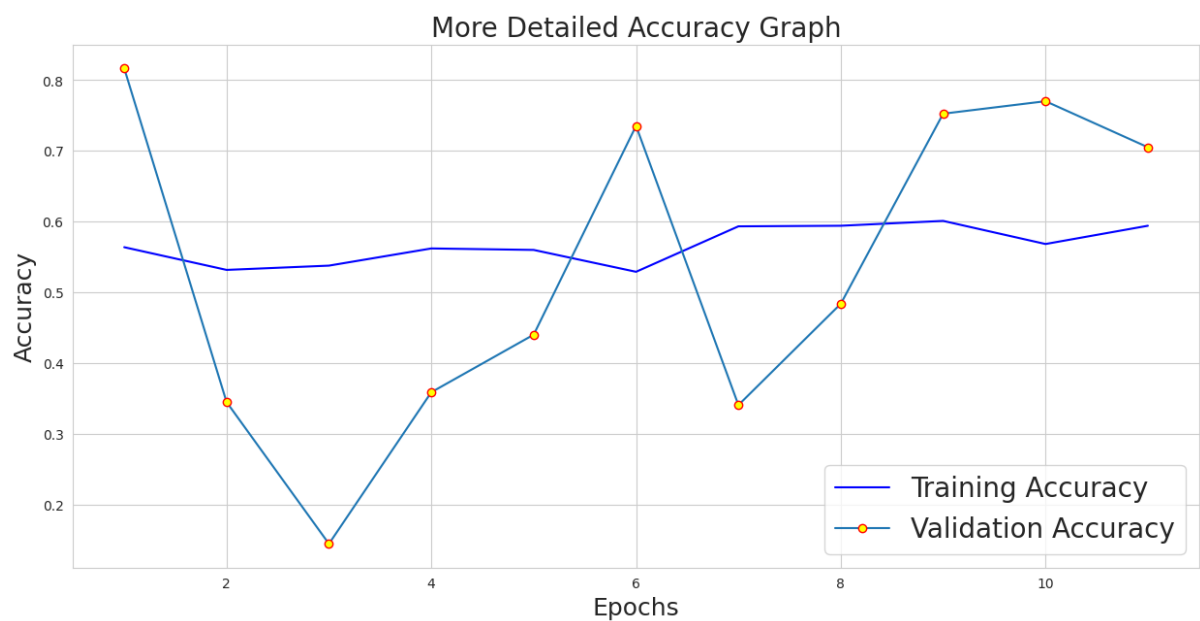
from sklearn.metrics import classification_report

print(classification_report(y_test, pred_log))
```

38/38 ————— 0s 7ms/step

	precision	recall	f1-score	support
0.0	0.90	0.79	0.84	1058
1.0	0.17	0.35	0.23	137
accuracy			0.74	1195
macro avg	0.54	0.57	0.54	1195
weighted avg	0.82	0.74	0.77	1195

4. Accuracy detailed graph



Lab 10

Task 1

1) Encoder Summary

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not
super().__init__(**kwargs)
Model: "sequential_3"
```

Layer (type)	Output Shape	Param #
flatten_1 (Flatten)	(None, 784)	0
gaussian_noise_1 (GaussianNoise)	(None, 784)	0
dense_10 (Dense)	(None, 400)	314,000
dense_11 (Dense)	(None, 200)	80,200
dense_12 (Dense)	(None, 100)	20,100
dense_13 (Dense)	(None, 50)	5,050
dense_14 (Dense)	(None, 25)	1,275

```
Total params: 420,625 (1.60 MB)
Trainable params: 420,625 (1.60 MB)
Non-trainable params: 0 (0.00 B)
None
```

2) Decoder Summary

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_4"
```

Layer (type)	Output Shape	Param #
dense_15 (Dense)	(None, 50)	1,300
dense_16 (Dense)	(None, 100)	5,100
dense_17 (Dense)	(None, 200)	20,200
dense_18 (Dense)	(None, 400)	80,400
dense_19 (Dense)	(None, 784)	314,384
reshape_1 (Reshape)	(None, 28, 28)	0

```
Total params: 421,384 (1.61 MB)
Trainable params: 421,384 (1.61 MB)
Non-trainable params: 0 (0.00 B)
None
```

3) Autoencoder Summary

```
# Your code to print the Autoencoder model
```

```
print(noise_remover.summary())
```

Model: "sequential_5"

Layer (type)	Output Shape	Param #
sequential_3 (Sequential)	(None, 25)	420,625
sequential_4 (Sequential)	(None, 28, 28)	421,384

Total params: 842,009 (3.21 MB)

Trainable params: 842,009 (3.21 MB)

Non-trainable params: 0 (0.00 B)

None

Task 2

1) Autoencoder Summary

```
autoencoder_2.summary()
```

Model: "sequential_5"

Layer (type)	Output Shape	Param #
sequential_3 (Sequential)	(None, 392)	1,320
sequential_4 (Sequential)	(None, 28, 28, 1)	155,953

Total params: 471,821 (1.80 MB)

Trainable params: 157,273 (614.35 KB)

Non-trainable params: 0 (0.00 B)

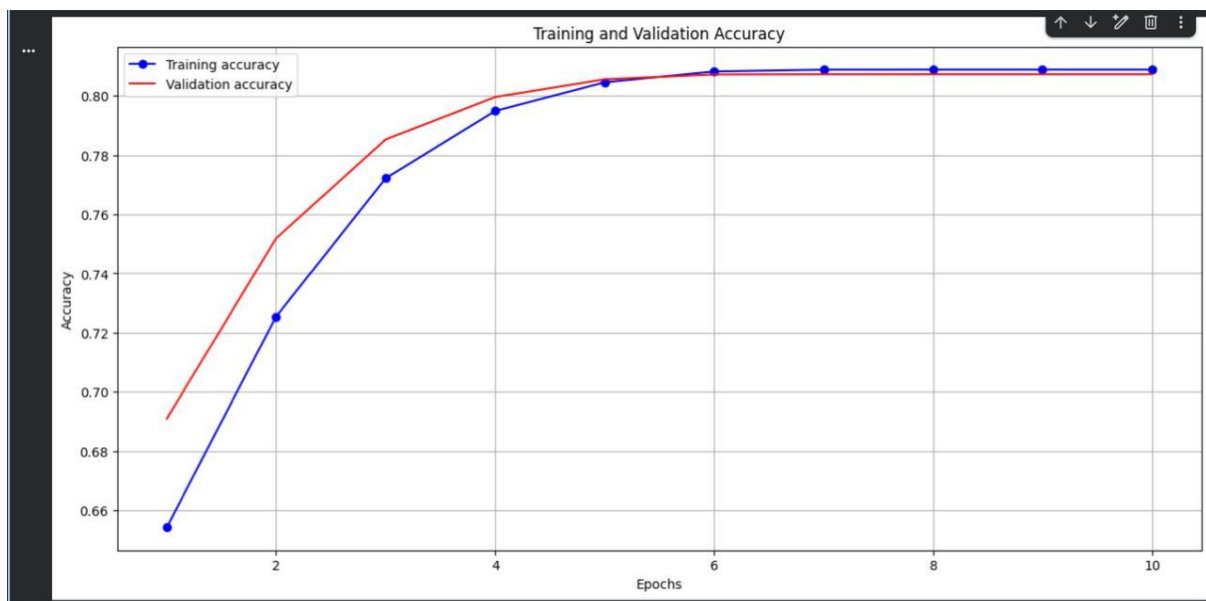
Optimizer params: 314,548 (1.20 MB)

2) Code of your autoencoder.fit()

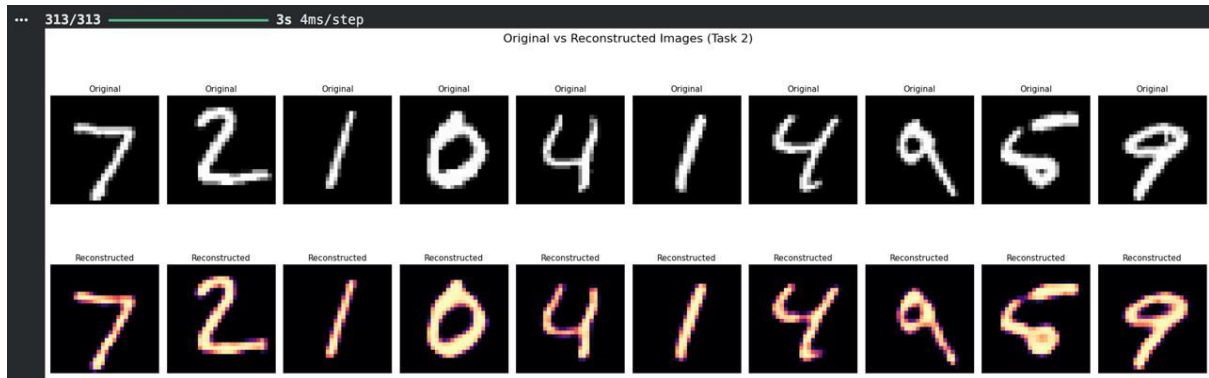
```
history_2 = autoencoder_2.fit(  
    x=x_train,  
    y=x_train,  
    epochs= Z + 40,  
    batch_size=128,  
    shuffle=True,  
    verbose=1,  
    validation_data=(x_test, x_test)  
)
```

```
Epoch 1/41  
469/469 ————— 74s 152ms/step - accuracy: 0.8087 - loss: 0.0310 - val_accuracy: 0.8087 - val_loss: 0.0310  
Epoch 2/41  
469/469 ————— 69s 125ms/step - accuracy: 0.8088 - loss: 0.0041 - val_accuracy: 0.8088 - val_loss: 0.0041  
Epoch 3/41  
469/469 ————— 62s 132ms/step - accuracy: 0.8088 - loss: 0.0040 - val_accuracy: 0.8088 - val_loss: 0.0040  
Epoch 4/41  
469/469 ————— 78s 125ms/step - accuracy: 0.8088 - loss: 0.0040 - val_accuracy: 0.8088 - val_loss: 0.0040  
Epoch 5/41  
469/469 ————— 57s 122ms/step - accuracy: 0.8088 - loss: 0.0040 - val_accuracy: 0.8088 - val_loss: 0.0040  
Epoch 6/41
```

3) Accuracy detailed graph



4) Document the plot of the decoded images for task 2. Change the colour, appearance, and shape of the plot according to your own choice (see `plt.figure()` parameters).



Lab 11

Task 1



Time for epoch 31 is 13.782390594482422 sec

Task 2 (Optional)

Epoch 30



Time for epoch 30 is 217.78 sec

Lab 12

1) calculation of your parameters

```
Z = 1
epochs_attn = Z + 50
patience_attn = Z + 10
```

2) your code of using EarlyStopping() and ModelCheckpoint() functions

```
early_stopping = EarlyStopping(
    monitor='val_loss',
    patience=patience_attn,
    restore_best_weights=True
)

model_checkpoint = ModelCheckpoint(
    filepath='best_attention_model.keras',
    monitor='val_loss',
    save_best_only=True,
    verbose=1
)
```

3) your code and training result using model.fit()

```
# Model 1: Simple RNN
history_1 = model_RNN.fit(
    trainX,
    trainY,
    epochs=40,
    batch_size=1,
    verbose=1,
    validation_data=(testX, testY)
)

# Model 2: Attention model (same practical session settings)
history_2 = model_attention.fit(
    trainX,
    trainY,
    epochs=40,
    batch_size=1,
    verbose=1,
    validation_data=(testX, testY)
)

# Model 3: Modified attention model with EarlyStopping + ModelCheckpoint
history_3 = model_attention_mod.fit(
    trainX,
    trainY,
    epochs=epochs_attn,
    batch_size=1,
    verbose=1,
    validation_data=(testX, testY),
    callbacks=[early_stopping, model_checkpoint]
)
```

```

Epoch 1/40
826/826 ————— 5s 4ms/step - loss: 0.0023 - val_loss: 0.0021
Epoch 2/40
826/826 ————— 3s 4ms/step - loss: 4.1405e-04 - val_loss: 0.0017
Epoch 3/40
826/826 ————— 3s 4ms/step - loss: 0.0010 - val_loss: 0.0016
Epoch 4/40
826/826 ————— 3s 4ms/step - loss: 0.0010 - val_loss: 0.0014
Epoch 5/40
826/826 ————— 3s 4ms/step - loss: 9.0255e-04 - val_loss: 0.0013
Epoch 6/40
826/826 ————— 3s 4ms/step - loss: 6.3360e-04 - val_loss: 0.0012
Epoch 7/40
826/826 ————— 3s 4ms/step - loss: 3.4606e-04 - val_loss: 0.0010
Epoch 8/40
826/826 ————— 3s 4ms/step - loss: 3.8129e-04 - val_loss: 9.1967e-04
Epoch 9/40
826/826 ————— 3s 4ms/step - loss: 0.0012 - val_loss: 8.1386e-04
Epoch 10/40
826/826 ————— 3s 4ms/step - loss: 1.6451e-04 - val_loss: 6.8800e-04
Epoch 11/40
826/826 ————— 3s 4ms/step - loss: 4.7511e-04 - val_loss: 5.6836e-04

Epoch 1: val_loss improved from inf to 0.00115, saving model to best_attention_model.keras
826/826 ————— 6s 5ms/step - loss: 0.0027 - val_loss: 0.0012
Epoch 2/51
814/826 ————— 0s 3ms/step - loss: 0.0023
Epoch 2: val_loss improved from 0.00115 to 0.00113, saving model to best_attention_model.keras
826/826 ————— 4s 4ms/step - loss: 0.0022 - val_loss: 0.0011
Epoch 3/51
814/826 ————— 0s 3ms/step - loss: 5.3263e-04
Epoch 3: val_loss improved from 0.00113 to 0.00110, saving model to best_attention_model.keras
826/826 ————— 4s 4ms/step - loss: 5.4502e-04 - val_loss: 0.0011
Epoch 4/51
818/826 ————— 0s 3ms/step - loss: 0.0022
Epoch 4: val_loss improved from 0.00110 to 0.00107, saving model to best_attention_model.keras
826/826 ————— 4s 4ms/step - loss: 0.0022 - val_loss: 0.0011
Epoch 5/51
820/826 ————— 0s 3ms/step - loss: 0.0019
Epoch 5: val_loss improved from 0.00107 to 0.00103, saving model to best_attention_model.keras
826/826 ————— 4s 4ms/step - loss: 0.0019 - val_loss: 0.0010
Epoch 6/51
819/826 ————— 0s 3ms/step - loss: 1.4200e-04
Epoch 6: val_loss did not improve from 0.00103
826/826 ————— 4s 4ms/step - loss: 1.5223e-04 - val_loss: 0.0013
Epoch 7/51
811/826 ————— 0s 3ms/step - loss: 0.0012
Epoch 7: val_loss improved from 0.00103 to 0.00095, saving model to best_attention_model.keras
826/826 ————— 3s 4ms/step - loss: 0.0012 - val_loss: 9.5222e-04

```

4) received loss for train and test datasets

```

# Model 1
train_mse_1 = model_RNN.evaluate(trainX, trainY)
test_mse_1 = model_RNN.evaluate(testX, testY)

# Model 2
train_mse_2 = model_attention.evaluate(trainX, trainY)
test_mse_2 = model_attention.evaluate(testX, testY)

# Model 3
train_mse_3 = model_attention_mod.evaluate(trainX, trainY)
test_mse_3 = model_attention_mod.evaluate(testX, testY)

print("Simple RNN - Train MSE =", train_mse_1)
print("Simple RNN - Test MSE =", test_mse_1)

print("Attention RNN - Train MSE =", train_mse_2)
print("Attention RNN - Test MSE =", test_mse_2)

print("Modified Attention RNN - Train MSE =", train_mse_3)
print("Modified Attention RNN - Test MSE =", test_mse_3)

```

```

26/26 ————— 1s 12ms/step - loss: 5.6704e-05
12/12 ————— 0s 22ms/step - loss: 5.0660e-05
26/26 ————— 1s 17ms/step - loss: 7.1430e-05
12/12 ————— 0s 32ms/step - loss: 3.0547e-05
26/26 ————— 1s 18ms/step - loss: 4.7832e-05
12/12 ————— 0s 35ms/step - loss: 4.8688e-06
Simple RNN - Train MSE = 4.78451038361527e-05
Simple RNN - Test MSE = 2.7386971851228736e-05
Attention RNN - Train MSE = 7.418636232614517e-05
Attention RNN - Test MSE = 1.2614154911716469e-05
Modified Attention RNN - Train MSE = 5.541241625905968e-05
Modified Attention RNN - Test MSE = 2.0242464415787254e-06

```

[46]:

```

import pandas as pd

ErrorMetrics = [
    [train_mse_1, test_mse_1],
    [train_mse_2, test_mse_2],
    [train_mse_3, test_mse_3]
]

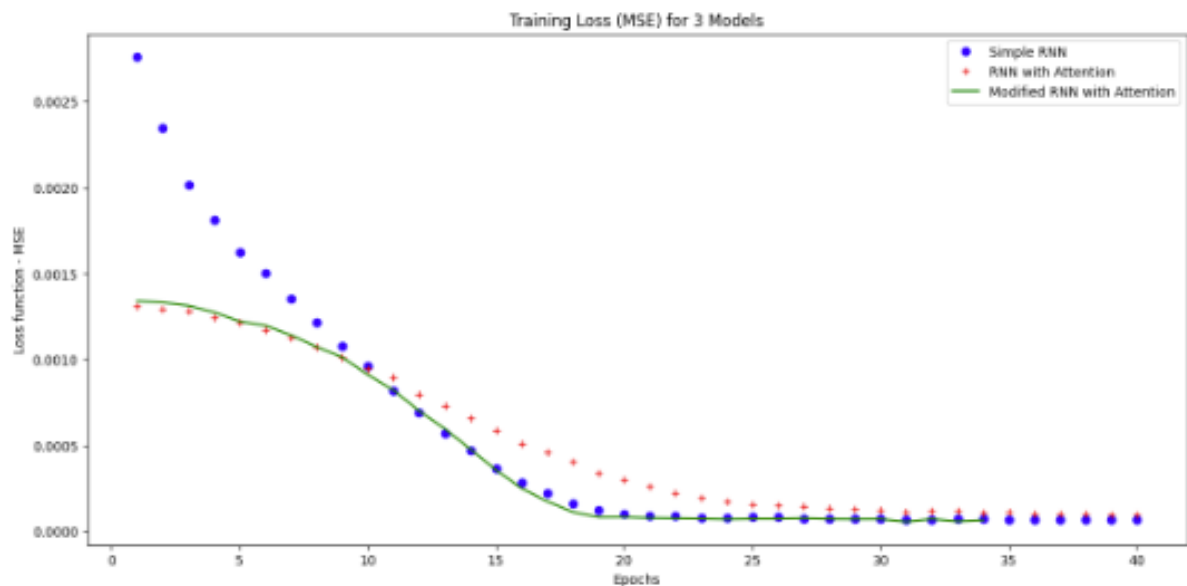
ErrorMetrics_pd = pd.DataFrame(ErrorMetrics)
ErrorMetrics_pd.columns = ['Train', 'Test']
ErrorMetrics_pd.index = [
    'Simple RNN',
    'RNN with Attention',
    'Modified RNN with Attention'
]

print(ErrorMetrics_pd)

```

	Train	Test
Simple RNN	0.000048	0.000027
RNN with Attention	0.000074	0.000013
Modified RNN with Attention	0.000055	0.000002

5) Loss (MSE) graph for the 3 models together during the training process



6) performance histogram for train and test loss (MSE) of the 3 models together

